A WAKKAQT FIELD DOCUMENT

WakkaQt: The Code Behind the Curtain

THE UNOFFICIAL PARTIAL COMPENDIUM

"No frameworks du jour. No microservices. No 'let's rewrite it in Rust'. Just C++17, FFmpeg, FFTW, an ONNX model, and a genuinely startling number of `QFile::rename()` calls that all had to be got exactly right."

COVERS VERSION 2.9.6

> *"No frameworks du jour. No microservices. No 'let's rewrite it in Rust'. Just C++17, FFmpeg, FFTW, an ONNX model, and a genuinely startling number of `QFile::rename()` calls that all had to be got exactly right."*

This is the companion nobody asked for to the manual everybody got: a tour of what's actually happening under WakkaQt's bonnet (sorry, hood — we'll be doing this in British English throughout, so brace yourself for the odd "colour" and "optimise"), for anyone curious enough to open the source tree and brave enough to keep reading past the first `#ifdef`.

We'll follow the same chapter order as the user manual, because consistency is a virtue and also because reinventing a table of contents is more work than this joke is worth. Each chapter pairs a plain-English explanation with real code, quoted verbatim, with file paths and line numbers so you can go and heckle it yourself.

And yes, "Partial" is doing double duty in that title. Read on to Chapter 08, and the word will start following you around the codebase like a small, honest ghost.

Table of Contents

---

00 — What Actually Happens When You Double-Click the Icon

Before any of the fun stuff (pitch correction! AI vocal removal! a pitch monitor with *opinions*!), the programme has to actually start up, and — like most software that touches audio hardware, video hardware, and the network all before its first window is visible — that turns out to be its own small adventure. Here's the whole of `main()`, roughly in the order it runs, for the benefit of anyone who has never had the pleasure of reading a `main.cpp` before:

1. **A couple of environment quirks get fixed before anything else exists.** On Windows, `QT_MEDIA_BACKEND` is forced to `ffmpeg`. Everywhere, `QT_FFMPEG_DECODING_HW_DEVICE_TYPES` is set to nothing at all — deliberately blank — because webcam footage comes back from the OS in a chroma format that hardware

video decoders choke on, and Qt only reads this variable once, at plugin load time, so it has to happen before a single `QApplication` object exists.

```
// src/main.cpp, lines 114–121
// Webcam recordings are baseline MJPEG in 4:4:4 chroma (yuvj444p), a
// combination VAAPI/CUDA hwaccel decoders on this FFmpeg backend can't
// initialize ("Failed to upload decode parameters: invalid parameter"),
// which spams errors and leaves the video preview blank. Must be set
// before QApplication/the FFmpeg plugin initializes...
qputenv("QT_FFMPEG_DECODING_HW_DEVICE_TYPES", "");
```

2. **`QApplication` is constructed**, at which point WakkaQt goes hunting for `frei0r` video-effect plugins on disk (needed for exactly one effect — the swirly 🌀 Vertigo filter — and nothing else) by trying a short list of well-known install paths per platform and setting `FREI0R_PATH` to the first one that actually exists. If none exist, nothing breaks; that one effect simply won't show up later. No plugin, no drama.
3. **A lock file is claimed** (`QLockFile` at `<tmp>/WakkaQt.lock`). If another instance already holds it, you get a polite dialogue box telling you WakkaQt is already running, and the new process quits with exit code 1. This is the whole of WakkaQt's "single instance" enforcement — no fancy IPC, just "whoever gets the lock file first wins".
4. **A rather elaborate `qInstallMessageHandler`** is wired in, which intercepts every `qDebug()` / `qWarning()` / `qCritical()` call anywhere in the codebase and turns it into a timestamped, colour-coded, occasionally-emoji-decorated line of rich text for the on-screen log widget — including a small roulette wheel of four different emoji for ordinary debug chatter (🔧 🐛 📝 🎶), picked by hashing the message text so the log doesn't read like the same wrench forever.
5. **Stale workspace directories from a previous crash get swept up** — but only ones more than a day old (see Chapter 08 for why that matters — a recent one might be a user's still-recoverable failed export).
6. **`MainWindow` is finally constructed**, the logger is wired up to it, and the window is shown. Only *now* does anything resembling "the app" appear on screen.

> 🍿 **LAYPERSON'S SUMMARY** Before you ever see a window, WakkaQt has already: patched two environment variables so your webcam doesn't upset FFmpeg, gone looking for an optional visual-effects plugin, made sure you're not accidentally running two copies of itself, wired up a very chatty logging system, and tidied its own bedroom from last time it crashed. All of that in well under a second, and all of it before the "SING" button has even been drawn.

---

01 — The Floor Plan

The source tree is organised into five directories, and — refreshingly for a project this age — the organisation is actually load-bearing, not just decorative. The build system (`CMakeLists.txt`) compiles each one into its own static library, with dependencies flowing strictly one way:

```
src/core   shared globals/types, session persistence, logging
src/dsp    pure DSP — vocal enhancement + vocal separation (MDX-Net)
src/media  multimedia infrastructure — FFmpeg wrapper, recording, playback
src/jobs   background-work QObjects (RenderJob, VocalSeparationJob, ...)
src/ui     MainWindow, dialogs, widgets
```

`wakkaqt_core` knows nothing about DSP or media (except an opt-in webcam-validation helper). `wakkaqt_dsp` is *allowed* to depend on `wakkaqt_media` (for its optional native-FFmpeg decode path) but never the other way round. `wakkaqt_jobs` sits on top of both, orchestrating the actual background work. `MainWindow` sits on top of everything, because somebody has to, and that somebody is always the UI layer in every codebase that has ever existed.

This matters more than it sounds like it should, because it's the reason the test suite in Appendix A was even possible to write without a full rewrite: the DSP and job layers were already separated cleanly enough from the GUI that they could be linked into a standalone test binary and pelted with fake data.

---

02 — Choosing Your Weapons, in Code

Chapter 1 of the manual shows you a friendly little dialogue with two tabs and a "Select" button. Underneath, it's `QMediaDevices::audioInputs()` and `QMediaDevices::videoInputs()`, dumped into two `QListWidget`s, with exactly one hard rule enforced on the way out:

```
// src/ui/mainwindowDevicesMgr.cpp, lines 311–321
// A microphone is required — WakkaQt can't record a performance without
// one. A camera is optional: WakkaQt falls back to audio-only recording,
// preview, and render when none is available.
if (audioInputs.isEmpty()) {
    QMessageBox::critical(this, "Device Error", "No audio devices available.");
    exit(-1);
    return;
}
if (videoInputs.isEmpty()) {
    qDebug() << "No camera detected; proceeding audio-only.";
}
```

No microphone, no karaoke — the app exits outright. No webcam, on the other hand, is treated as a perfectly ordinary life choice: a disabled placeholder row appears reading *"No camera detected — will record audio-only"*, and the entire rest of the application quietly reconfigures itself around a `bool hasCamera` flag rather than throwing errors at you for the crime of not owning a camera.

There's a nice bit of unsung diligence in `configureMediaComponents()` too: when picking which camera *format* to actually use, WakkaQt scores the available options and actively prefers raw, uncompressed pixel formats over ones the camera has already JPEG-compressed internally — on the reasoning that recompressing a frame that's already been lossily squashed once is a false economy. It'll only settle for JPEG if that's genuinely all the camera offers.

> 🍿 **WHY THIS MATTERS** Recording your own face is embarrassing enough at full quality. WakkaQt at least tries not to make it *blurrier* than it has to be.

03 — The Main Stage: MainWindow and Its Seven Moods

`MainWindow` is, structurally speaking, the largest room in the house — split across half a dozen `mainwindow*Mgr.cpp` files (`RecorderMgr`, `RenderMgr`, `SeparatorMgr`, `PlaybackMgr`, `LibraryMgr`, `DevicesMgr`) purely so no single file has to be three thousand lines long, though they're all still the very same C++ class underneath, sharing the very same private member variables. Splitting files is not the same thing as splitting responsibilities, and the codebase is entirely honest with itself about that in its own comments.

What genuinely does hold the whole thing together is a small, seven-value state machine:

```
// src/ui/mainwindow.h, lines 123–131
enum class State {
    Idle,         // nothing in progress — recording/library/separation may start
    Recording,    // camera/audio actively recording
    Aborting,     // transient: user aborted, stopRecording() takes the discard branch
    Finalizing,   // post-stop work + output-path/resolution/PreviewDialog flow (live
        record)
    Restoring,    // same flow, driven by a restored library session instead
    Rendering,    // RenderJob active — reached from Finalizing or Restoring
    Separating,   // generateBackingTrack() vocal separation in progress
};
```

Every single transition between these seven states has to pass through one gatekeeper method, `trySetState()`, which consults a lookup table of exactly which transitions are legal and simply refuses (with a logged warning, not a crash) to perform any that aren't:

```
// src/ui/mainwindow.cpp, lines 731–756
bool MainWindow::trySetState(State next)
{
    if (next == m_state)
        return true;

    static const QMap<State, QSet<State>> kAllowed = {
        { State::Idle,       { State::Recording, State::Restoring, State::Separating }
        },
        { State::Recording,  { State::Aborting, State::Finalizing, State::Idle } },
        { State::Aborting,   { State::Idle } },
        { State::Finalizing, { State::Rendering, State::Idle } },
        { State::Restoring,  { State::Rendering, State::Idle } },
        { State::Rendering,  { State::Idle } },
        { State::Separating, { State::Idle } },
    };
    if (!kAllowed.value(m_state).contains(next)) {
        qWarning() << "MainWindow: rejected illegal state transition"
                   << int(m_state) << "->" << int(next);
        return false;
    }
    m_state = next;
    return true;
}
```

The pay-off of this small table is disproportionate to its size: "start a vocal separation while a recording is in progress" isn't a bug that somebody has to remember not to write and occasionally forgets — it's a structural impossibility, rejected the moment it's attempted, by one piece of code that every single feature has to go through. No feature gets to invent its own rules about what it's allowed to interrupt.

Two things are deliberately **not** part of this state machine, and the source comments are refreshingly upfront about why: `hasCamera` (is a camera *currently plugged in*) and `recordingHasWebcam` (did *this particular session* actually have webcam footage) are readiness/data concerns, not lifecycle phases — conflating "is a camera plugged in right now" with "does the recording I'm about to render have a webcam track" would let you restore an old audio-only session while a webcam happens to be plugged in and have it wrongly acquire video, or vice versa. It's a small distinction that would be very easy to get quietly, invisibly wrong.

As for what the constructor actually does on the way to that first `w.show()`: it builds the menu bar and an About dialog first — the About dialog doubles rather charmingly as an in-source changelog/feature poster, with lines like `🤖 AI vocal separation → backing track generator (UVR-MDX-NET-Inst_HQ_3, runs 100% offline via ONNX Runtime)` sitting in a raw HTML string literal in the middle of `mainwindow.cpp` — then the video widget and placeholder logo, then only *after* all of that does it start touching audio/video hardware at all. WakkaQt draws its own face before it's capable of hearing or seeing anything.

---

04 — Feeding the Beast

Getting a song onto the screen has, per the manual, three routes: open a local file, browse YouTube properly, or paste a link and mash FETCH. In code, routes one and three both converge on a plain old `QFileDialog` / `QUrl` check; route two is where things get properly theatrical.

Local files are the boring, correct way to do it — one `QFileDialog` with every supported extension in its filter, straight into `playVideo()`. **The lazy URL route** validates that what you pasted is a *single video* link, not a playlist, before doing anything else:

```
// src/ui/mainwindowPlaybackMgr.cpp, lines 217–232
void MainWindow::fetchVideo() {
    const QString urlStr = urlInput->text().trimmed();
    ...
    const QUrl url(urlStr);
    if (!isSingleYouTubeVideoUrl(url)) {
        QMessageBox::warning(
            this, "Invalid URL",
            "Please paste a single *video* URL from YouTube.\n"
            "Tip: Use YouTube's \"Share\" button and copy that link.\n"
            "Playlists are not supported.");
        return;
    }
```

The YouTube browser, though, is where the whole "AI-powered karaoke search" fantasy dissolves into something wonderfully mundane. There is no karaoke-detection algorithm. There is no machine learning. There are two separate `yt-dlp` subprocesses, launched in parallel, one of which has the literal word `"karaoke"` bolted onto the end of your search query:

```
// src/ui/youtubesearchdialog.cpp, lines 293–302
m_karaokeProc = new QProcess(this);
m_karaokeProc->start("yt-dlp", QStringList()
    << QString("ytsearch%1:%2 karaoke").arg(kPageSize).arg(q) << baseArgs);
connect(m_karaokeProc, &QProcess::finished, this,
        &YoutubeSearchDialog::onKaraokeDataReady);

m_originalsProc = new QProcess(this);
m_originalsProc->start("yt-dlp", QStringList()
    << QString("ytsearch%1:%2").arg(kPageSize).arg(q) << baseArgs);
```

That's the entire "Karaoke versions on top, Originals below" feature from Chapter 3 of the manual: string concatenation, twice, run concurrently, with the results parsed line-by-line out of `yt-dlp`'s `--dump-json` NDJSON output. Thumbnails are fetched directly from YouTube's own static CDN (`img.youtube.com/vi/<id>/mqdefault.jpg`) with an ordinary `QNetworkAccessManager` request per card. It works precisely as well as searching YouTube for "song name karaoke" yourself would — because that is, quite literally, what it's doing, just faster and in a nicer window.

The download flow, once you've picked something, is a small two-act play:

```
// src/ui/DownloadDialog.cpp, lines 124–133
void DownloadDialog::onDownloadStdErr() {
    const QString chunk = QString::fromUtf8(m_downloadProc->readAll());
    static QRegularExpression re(R"((\d{1,3}(?:\.\d+)?)%)");
    QRegularExpressionMatchIterator it = re.globalMatch(chunk);
    double lastPercent = -1.0;
    while (it.hasNext()) {
        auto m = it.next();
        lastPercent = m.captured(1).toDouble();
    }
    if (lastPercent >= 0.0)
        m_progress->setValue(static_cast<int>(lastPercent + 0.5));
```

Act One runs `yt-dlp --print filename` to *predict* the eventual filename (and sanity-checks it against a character whitelist) without downloading anything. Act Two runs the real download and regex-scrapes the very last `NN.N%` substring out of whatever text `yt-dlp` happened to print to its terminal in the most recent output chunk, to drive an actual `QProgressBar`. There is no structured progress API on offer here — just a CLI tool's scrolling text, grepped for a percentage sign, same as a human would do watching the terminal themselves.

> 🍿 **A CONFESSION THE MANUAL DOESN'T MAKE** Every "smart" download feature in this chapter is, underneath, `yt-dlp` doing all the actual work, and WakkaQt doing the very respectable job of building a nice window around it, parsing its output, and getting out of the way.

---

05 — Showtime: What SING Actually Does

Pressing 🎤 SING triggers `startRecording()`, and the "perfect sync" the manual promises turns out to be refreshingly unglamorous in its opening move: rewind the playback to zero, start the audio recorder, start the video recorder (if there is one), then hit play — all within a handful of statements, one after another, as fast as the CPU can execute them:

```
// src/ui/mainwindowRecorderMgr.cpp, lines 46–84 (abridged)
recordingHasWebcam = hasCamera;
...
vizPlayer->seek(0, true);
player->pause();
...
audioRecorder->startRecording(audioRecorded); // start audio recorder
if (hasCamera && mediaRecorder) {
    mediaRecorder->record();                  // start recording video
} else {
    onRecorderStateChanged(QMediaRecorder::RecordingState);
}
player->play(); // start the show
```

The genuinely clever synchronisation work doesn't happen at the start at all — it happens when you stop. `stopRecording()` measures how long each recorded file actually turned out to be against where playback had actually got to, and derives the audio/video timing offsets *from that*, rather than trusting a naive stopwatch running alongside the recording — the source comments explicitly note that a stopwatch-based approach silently diverges from reality the moment your system has above-average latency. The SING button itself doesn't even change identity to do this — it just relabels itself "Finish!" and calls the very same handler again to stop, same button, opposite verb, no separate code path to keep in sync with the first one.

Underneath the audio recorder is a small trick worth admiring purely for its cheek:

```
// src/media/audiorecorder.cpp, lines 95–98
// Reserve the 44-byte WAV header up front with placeholder (zero) size
// fields, so raw PCM streams in directly behind it as QAudioSource
// writes. stopRecording() then only has to patch the two size fields...
writeWavHeader(m_outputFile, m_audioFormat, 0, QByteArray());
```

Rather than buffering an entire performance in memory and writing a proper header once the final size is known, WakkaQt writes a *lying* WAV header the instant recording starts — one that claims a data size of zero — then streams raw PCM straight onto disk behind it as it arrives from the microphone. When you finally stop singing, it doesn't rewrite the file; it seeks back to exactly two four-byte fields near the start and overwrites just those, now that it finally knows the truth. It's the audio-file equivalent of writing "TBC" on a cheque and coming back to fill in the amount later.

Meanwhile, the pitch monitor strip watching over the top of the window the entire time — recording or not — is running a from-scratch implementation of the **YIN algorithm**: pick the loudest 2048-sample block, compute a difference function against time-shifted copies of itself, normalise it into a cumulative mean, find the best-matching lag, then refine that lag with a parabolic interpolation for sub-sample accuracy, clamped to a plausible 50–1200 Hz singing range. The result gets painted green within 15 cents of the target note, yellow within 30, and red beyond that — "no mercy," as the app's own About box rather smugly puts it.

---

06 — The Karaoke Studio: Sliders, Signals, and a Progress Bar That Fibs Politely

The Karaoke Studio — the Preview dialog, dressed up in a nicer title — is essentially a thin, honest UI wrapped around `VocalEnhancer`, a roughly 1,700-line DSP engine that does pitch correction, noise reduction, formant preservation, reverb, and mastering, all built on FFTW plans created

once and reused for the entire session. Every slider on the Vocal Tuning tab is wired to a plain member variable with almost embarrassingly little ceremony:

```
// src/ui/previewdialog.cpp, lines 344–348
connect(reverbRoomSlider, &QSlider::valueChanged, this, [this](int v) {
    m_reverbRoomSize = v / 100.0;
    reverbLabel->setText(QString("Reverb — Room: %1%  Decay: %2%  Mix: %3%")
        .arg(v).arg(int(m_reverbDecay*100)).arg(int(m_reverbMix*100)));
});
```

Ten-odd sliders and combo boxes like this feed straight into a `PreviewJob::EnhanceParams` struct, and pressing **Apply Enhancement** hands the whole bundle over to a background job:

```
// src/ui/previewdialog.cpp, lines 729–745 (abridged)
PreviewJob::EnhanceParams params;
params.pitchCorrectionAmount = pitchCorrectionAmount;
...
progressTimer->start(55);
if (!previewJob->enhance(previewInputAudioData, format, params)) {
    // Already busy re-processing a previous request — remember to run
    // this one (with whatever the sliders read at that time) once it's done.
    pendingPreviewRebuild = true;
}
```

That progress bar you watch crawl across the screen isn't being told anything by the DSP engine in real time — nobody's emitting proper progress signals from deep inside a phase-vocoder loop. Instead, a `QTimer` fires every 55 milliseconds and simply *asks* the enhancer how far along it reckons it is (`getProgress()`), a genial little white lie of a UI pattern that works perfectly well provided nobody looks too closely.

Underneath those sliders, `VocalEnhancer` is doing rather more than the friendly labels let on:

- **Pitch correction** is phase-vocoder pitch shifting — frequency-domain manipulation of a 2048-sample FFT window, snapped towards whichever note in your chosen key/scale is nearest, at a strength and retune speed you control.
- **Formant preservation** runs an LPC (linear predictive coding) analysis to separate your voice's spectral *envelope* (which makes a voice sound like *your* voice) from its pitch, so that shifting the pitch aggressively doesn't also drag your formants along with it and leave you sounding like a chipmunk mid-panic.
- **Noise reduction** is a spectral-subtraction gate with an *adaptive* noise floor — it spends a short window at the start of the take learning what "silence" sounds like in your specific room, rather than assuming a fixed threshold.
- **Reverb** is a Freeverb-style Schroeder reverb — the same lineage of comb-filter/all-pass-filter algorithm that's been quietly powering "make this sound like a room" effects since the 1990s.

The Effects tab, meanwhile, doesn't hand-code twelve separate UI panels for its twelve video effects. It generates them: each entry in a shared `videoEffectPresets` table declares its own parameters (min, max, default, decimal places), and the dialogue builds one generic slider per parameter, mapped from a 0–1000 integer range onto whatever real-valued range that effect actually wants. Adding a thirteenth effect to WakkaQt is a data-table entry, not a new dialog.

---

07 — The Backing Track Sorcery: Teaching a Neural Network to Un-sing Someone

This is the one feature in the whole app that genuinely involves a neural network, and — true to the manual's "no cloud, no subscription" pitch — it's a small (≈80 MB) pre-trained MDX-Net model (`UVR-MDX-NET-Inst_HQ_3`) run entirely locally through the ONNX Runtime, downloaded exactly once and checked against a pinned SHA-256 hash before WakkaQt will trust it. `VocalSeparator::separate()` runs the model over a full STFT/iSTFT pipeline built on the very same FFTW library doing the vocal-enhancement work in Chapter 06.

The interesting engineering here, though, isn't the neural network itself — it's everything wrapped *around* it, in `VocalSeparationJob`, to make a two-phase, genuinely-slow, genuinely-can-go-wrong-in-several-places operation feel calm and recoverable from the outside. Separation and export are deliberately two separate phases:

```
// src/jobs/vocalseparationjob.h, lines 32–46 (abridged)
// Phase 1: runs VocalSeparator::separate() on a worker thread, writing
// into a private per-run workspace directory this job creates and owns...
void separate(const QString &inputFile);
void cancelSeparate();

// Phase 2: mux/transcode the separated WAV into its final destination.
void exportResult(const ExportParams &params);
void cancelExport();
```

Why bother separating them? Because Phase 1 (the actual AI inference) is the expensive part, and Phase 2 (muxing the result back onto video, or transcoding it to MP3) is comparatively cheap but has more ways to fail — wrong output format, disk full, a destination folder that's gone away. If Phase 2 fails, the code goes out of its way **not** to discard the Phase 1 result:

```
// src/jobs/vocalseparationjob.h, lines 51–58
// Not called automatically on export failure: exportFailed()'s error
// message preserves the workspace path on purpose so a failed mux/encode
// never costs the user having to re-run the (expensive) separation.
void discardWorkspace();
```

So if muxing the separated instrumental back onto a video file fails, WakkaQt doesn't shrug and make you wait through the neural network again — it offers you Try Again, Save as WAV instead, or Open the Folder yourself, because the actual expensive result is still sitting safely on disk, exactly where it was left. Cancellation is cooperative on the native FFmpeg path (a shared `std::atomic<bool>` checked periodically inside the mux/encode loop) and brutally direct on the fallback path (the plain `ffmpeg` CLI has no "please stop nicely" flag, so WakkaQt just kills the process). Either way, whatever got partway written is cleaned up rather than left as a broken file.

---

08 — Rendering Your Masterpiece: FFmpeg, Twice Over

`RenderJob` has two entirely separate implementations of "produce the final video" behind one identical public interface: a **native** path that calls into FFmpeg's own C libraries directly (`libavformat`, `libavcodec`, `libavfilter`, …) from inside the process, and a **fallback** path that simply spawns the `ffmpeg` command-line tool as a subprocess and scrapes its `stderr` for

a `time=HH:MM:SS.CS` string to estimate progress — chosen at *compile time* depending on whether the FFmpeg development headers were available when WakkaQt itself was built.

Both paths share one very deliberately-designed habit: **never write directly to the file the user actually asked for.**

```
// src/jobs/atomicfilecommit.h, lines 6–25 (abridged)
// FFmpeg infers a file's output container/codec from its *last* extension,
// so a sidecar name must preserve it — "song.mp4" -> "song.partial.mp4",
// not "song.mp4.partial" (which FFmpeg would read as a ".partial" file and
// fail to find a muxer for).
QString sidecarPathFor(const QString &finalPath, const QString &tag);

// Replaces finalPath with the already-written, already-validated file at
// partialPath. If finalPath exists, it's moved aside to a backup sidecar
// *before* the partial is renamed into place, so a failure partway through
// restores the previous file instead of losing it...
QString commitPartialOverFinal(const QString &partialPath, const QString &finalPath);
```

Both `RenderJob` and `VocalSeparationJob` render into a `song.partial.mp4`-style sidecar sitting right next to the real destination, and only ever touch the real filename at the very end, atomically, once the render has actually, verifiably succeeded. If it fails or you press Abort Render halfway through, the sidecar gets deleted and the file you already had — if there was one — is never so much as glanced at. This is the exact mechanism the manual's "Abort Render stops it cleanly without leaving a broken file behind" line is describing, and it is, refreshingly, telling the truth.

The rendered video itself carries one more nice touch worth a mention: the pitch-indicator strip burned into the corner of every frame isn't drawn from the *corrected* audio — it's drawn from the **raw**, unprocessed vocal take, so the coloured note you see in the final video is an honest record of how you actually sang, not a flattering readout of how the DSP pipeline made you sound afterwards.

---

09 — The Session Library: Everything, Saved Transactionally

Every performance gets written to `~/.WakkaQt/library/<uuid>/`, and `SessionRepository::saveSession()` follows the same "build it somewhere safe first, then commit" philosophy as the renderer:

```
// src/core/sessionrepository.cpp (paraphrased structure)
const QString partialDir = libraryRoot() + "/" + id + ".partial";
const QString finalDir   = libraryRoot() + "/" + id;
// ... copy audio.wav, webcam.mkv (if any), playback.wav, write
// offsets.json and session.json into partialDir ...
if (!dir.rename(partialDir, finalDir)) { /* abort, nothing was ever "half-saved" */ }
```

A session only ever exists in the library as a fully-formed, fully-validated directory — right up until the final `rename()`, it's sitting in a `.partial` folder nobody else is looking at, so a crash or a lost-power moment mid-save can never leave a half-written entry cluttering your archive. Restoring one is, structurally, the same trick played in reverse: `restoreSession()` doesn't just check that `audio.wav` and `playback.wav` *exist* — it actually opens and parses them as real

WAV files, because a truncated file can still have a perfectly respectable non-zero size while containing nothing usable.

The genuinely charming bit is what happens *after* a successful restore: `restoreAndRender()` repoints the very same temporary-file globals a live recording would use at the restored session's files, then walks straight back into the exact same save-dialogue → resolution question → Karaoke Studio → render pipeline a live performance ends up in. As far as the rest of the application is concerned, resuming a session from three weeks ago and having just finished singing it thirty seconds ago are the same event. There is no separate "restored session" code path to quietly drift out of sync with the live one, because there isn't a second one to drift.

Appendix A — The Quality Control Department

For a long stretch of its life, this project had precisely zero automated tests — everything from WAV parsing to atomic file commits to the entire render pipeline was validated the old-fashioned way, by a human clicking buttons and hoping. That changed rather thoroughly in the run-up to version 2.9.6, and since we happen to have been in the room for it, it seemed rude not to mention it here.

The trick that made testing `RenderJob` and `VocalSeparationJob` possible at all, without needing a real copy of FFmpeg or a genuine 80 MB neural network sitting in CI, was to notice that both classes only ever call into "the expensive bit" **once per operation** — not once per video frame, not once per audio sample. That meant a test-only substitute could be injected at exactly that one call site, at essentially zero runtime cost in production, because production simply never sets it:

```
// src/jobs/renderjob.h (abridged)
using Engine = std::function<bool(const Params &params,
                                  const QString &partialOutputPath,
                                  const std::function<void(double)> &progressCb,
                                  const std::atomic<bool> *cancelled)>;
void setEngineForTesting(Engine engine) { m_testEngine = std::move(engine); }
```

With that one seam in place, tests can now assert things that used to be entirely a matter of faith — for instance, that cancelling a render genuinely never touches a file that already existed at the destination:

```
// tests/test_renderjob.cpp (abridged)
void cancelledRender_leavesPreexistingOutputFileUntouched()
{
    RenderJob job;
    job.setEngineForTesting([](const RenderJob::Params &, const QString
        &partialOutputPath,
                               const std::function<void(double)> &,
                               const std::atomic<bool> *cancelled) {
        // ...writes some "partial garbage", then waits for cancellation...
        for (int waited = 0; waited < 2000; waited += 10) {
            if (cancelled && cancelled->load()) return false;
            QThread::msleep(10);
        }
        return true;
    });
    // ... pre-populate outputPath with "precious pre-existing render" ...
    job.start(makeParams(outputPath));
    job.cancel();
    // ... assert the pre-existing file is still there, byte for byte ...
}
```

That particular test exists because this is exactly the bug that used to lurk in the UI layer: a leftover `QFile::remove(outputFilePath)` from before the atomic-commit rework, quietly ready to delete a user's valid existing file the moment they cancelled a re-render over it. It was found and fixed in the course of writing this very documentation companion, which feels like an appropriately on-brand way for a project this candid about its own code to have discovered a bug.

There are now six test suites, covering: atomic file commits (including the exact "unremovable leftover backup" failure mode), WAV chunk parsing (including malformed and extension-chunk cases real encoders actually produce), session save/restore round-trips, `VocalEnhancer`'s clamped parameter setters, and the full cancellation/atomic-commit/failure-recovery behaviour of both `RenderJob` and `VocalSeparationJob` — all of it running in about a second and a half, with no FFmpeg binary, no ONNX model, and no human required.

Appendix B — The Build System's Family Tree

`CMakeLists.txt` compiles the source tree from Chapter 01 into a small hierarchy of static libraries, each one only depending on what it genuinely needs:

```
wakkaqt_core  ← wakkaqt_media (only if FFmpeg dev libs found)
wakkaqt_dsp   ← wakkaqt_media (only if FFmpeg dev libs found)
wakkaqt_jobs  ← wakkaqt_core, wakkaqt_dsp, wakkaqt_media (conditionally)
WakkaQt (exe) ← all of the above
```

Two features are entirely optional at build time, and the project treats "the dependency isn't installed" as a perfectly ordinary outcome rather than a fatal error: no FFmpeg development headers means native rendering quietly falls back to spawning the `ffmpeg` CLI instead (see Chapter 08); no ONNX Runtime means the 🎵 Backing Track button simply never appears in the UI at all, rather than appearing and then failing when clicked. Both are detected at `cmake` configure time and threaded through as preprocessor definitions (`WAKKAQT_FFMPEG_NATIVE`, `WAKKAQT_ONNX`) that the rest of the codebase checks with plain `#ifdef`s.

And that's the lot. If the user manual left you thinking WakkaQt was held together by confidence and good intentions, hopefully this has at least demonstrated that it's held together by confidence, good intentions, and a genuinely unreasonable number of `std::atomic<bool>` *flags being checked in tight little loops. Go and read the actual source. It's rather good company.*